

Using LLMs for Writing ATL Model Transformation Code

Saqib Iqbal

College of Engineering

Al Ain University

Al Ain, United Arab Emirates

saqib.iqbal@aau.ac.ae

Issam Al-Azzoni

College of Engineering

Al Ain University

Al Ain, United Arab Emirates

issam.alazzoni@aau.ac.ae

Abstract—Large Language Models (LLMs) have demonstrated remarkable capabilities in code generation across various programming languages and domains. In this paper, we present the first empirical study investigating the use of LLMs for generating ATL (ATLAS Transformation Language) model transformation code in the context of Model-Driven Engineering (MDE). Through controlled experiments using minimal prompts and metamodels, we show that state-of-the-art LLMs such as ChatGPT and Claude can produce syntactically correct and executable ATL transformations. The results confirm that LLMs have implicitly learned ATL's structural and semantic patterns, enabling the automatic generation of valid transformation rules. This opens new research directions for the MDE community, suggesting that LLMs can serve as practical assistants or co-authors in building and validating model transformation solutions.

Index Terms—ATL, LLMs, Model Transformation, Model-Driven Engineering

I. INTRODUCTION

Model-Driven Engineering (MDE) converts abstract specification to implementable artifacts by exploiting precise and executable model transformations. ATL (ATLAS Transformation Language) is one of the most widely used transformation languages which is a hybrid declarative/imperative language to produce target models from source models [1]. In parallel, large language models (LLMs) have shown strong capabilities in code synthesis, debugging and interoperability in multiple platforms and have shown great prospect in domain-specific languages. Early works such as Codex demonstrated that the model transcribed on large code corpora can generate accurate and testable models / programs from natural language prompts [2]. Some surveys have demonstrated the potential, open problems, reliability and evaluation issues and specification handling related to code generation [3].

This paper explores focused research on the extent to which off-the-shelf LLMs can be exploited for correct ATL transformations from a set of minimal prompts and metamodels. Concretely, we have evaluated Claude, ChatGPT and DeepSeek using Ecore metamodels and unambiguous prompts. We have checked syntactic correctness (compilations), execution on the ATL VM and semantic adequacy of the produced target model.

Our contributions include: (1) a compact evaluation protocol for ATL code generation from LLMs (2) an empirical comparison of the three contemporary LLMs on a standard

ATL task, including evaluation of compilation/execution errors and formatting inconsistencies, and (3) discussion on the implications for exploiting LLMs in MDE domain.

The remainder of this paper is structured as follows. Section II discusses the related work. Section III illustrates the approach, and Section IV discusses the evaluation of the approach. Finally, Section V concludes the paper and discusses future work.

II. RELATED WORK

This section provides a summary of existing work in this domain. Model transformation is a vital step in MDE, with taxonomies organizing languages by execution semantics, constraints and tooling [4]. Czarnecki and Helsen (2006) has provided a comprehensive survey based on features that remains the touchstone in this field [5]. ATL itself is rule-based and QVT-like approach which has been around for a long time with eclipse integration [1]. Specialized LLMs on code, such as Codex has already demonstrated that natural-language prompts can yield executable programs with the provision of evaluation and functional-correctness checks like HumanEval [2]. Some recent surveys have also highlighted challenges such as hallucination and ambiguous specifications [3].

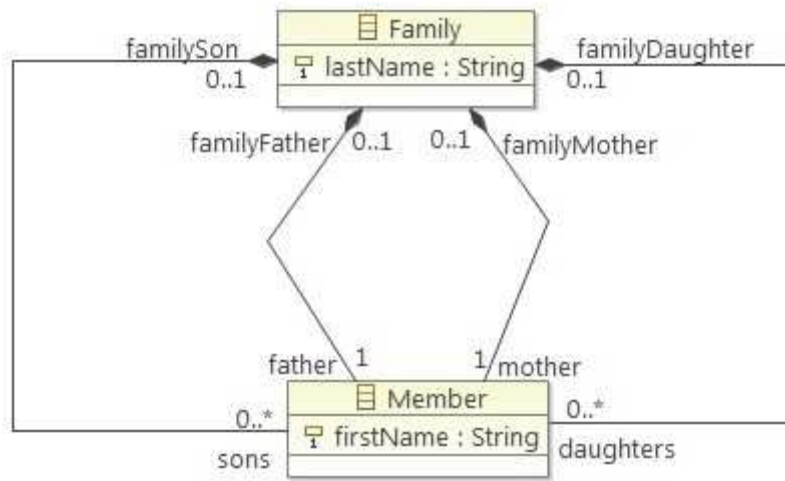
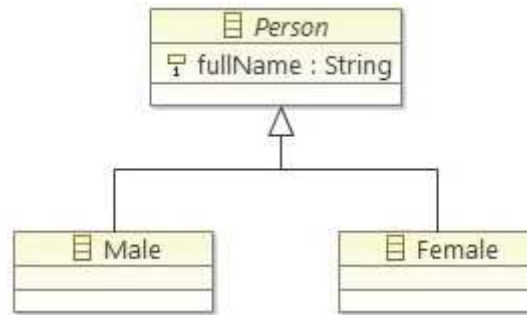
Although research related to connecting LLMs with MDE is still an emerging field, initial contributions suggest potential in this merge in the domains of model synthesis, transformation authoring and evolution. For example, Martinez et al. (2013) [6] explored reactive model techniques, while Tisi et al. (2011) [7] proposed lazy execution strategies. These are the concepts that could be combined with LLMs to validate or improve model transformation.

III. APPROACH

We present our approach on well-known basic example on ATL: *Families2Persons*¹. The objective of this example is to perform a transformation from a list of families to a list of persons.

Figure 1 shows the *Families* metamodel. A *Family* is composed of several members: the father, the mother, the sons,

¹https://wiki.eclipse.org/ATL/Tutorials/_Create_a_simple_ATL_transformation#The_ATL_transformation_code

Fig. 1. The *Families* metamodelFig. 2. The *Persons* metamodel

and the daughters. A *Family* has a single attribute which is the family's last name. A *Member* has a single attribute which is the first name.

In the *Families* metamodel, every composition relationship has labels at both ends. For example, if a *Member* acts as a father of a *Family*, then this member's *familyFather* references the *Family*. This is analogous to the other cases: mother, sons, and daughters.

Figure 2 shows the *Persons* metamodel. A *Person* is an abstract class with two sub-types: *Male* and *Female*. This expresses that a person can be male or female. A person has a single attribute *fullName* which represents the person's full name.

The *Families2Persons* transformation transforms *Families* model instances (containing families and their information) into *Persons* model instances (containing persons which could be male or female). The full name is determined by concatenating a member's first name with his/her family's last name. The gender of a person should be inferred based on whether the member is a father or a son (in this case, the person is male) or a mother or daughter (in this case, the person is female). For example, Consider the following *Families* model instance (in XMI):

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xmi:XMI xmi:version="2.0"
  xmlns:xmi="http://www.omg.org/XMI" xmlns="Families">
  <Family lastName="March">
    <father firstName="Jim"/>
    <mother firstName="Cindy"/>
    <sons firstName="Brandon"/>
    <daughters firstName="Brenda"/>
  </Family>
  <Family lastName="Sailor">
    <father firstName="Peter"/>
    <mother firstName="Jackie"/>
    <sons firstName="David"/>
    <sons firstName="Dylan"/>
    <daughters firstName="Kelly"/>
  </Family>
</xmi:XMI>

```

```

<father firstName="Jim"/>
<mother firstName="Cindy"/>
<sons firstName="Brandon"/>
<daughters firstName="Brenda"/>
</Family>
<Family lastName="Sailor">
  <father firstName="Peter"/>
  <mother firstName="Jackie"/>
  <sons firstName="David"/>
  <sons firstName="Dylan"/>
  <daughters firstName="Kelly"/>
</Family>
</xmi:XMI>

```

The corresponding *Persons* model instance which is correctly generated by the *Families2Persons* model transformation is:

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xmi:XMI xmi:version="2.0"
  xmlns:xmi="http://www.omg.org/XMI" xmlns="Persons">
  <Male fullName="Jim March"/>
  <Male fullName="Brandon March"/>
  <Male fullName="Peter Sailor"/>
  <Male fullName="David Sailor"/>
  <Male fullName="Dylan Sailor"/>
  <Female fullName="Cindy March"/>
  <Female fullName="Brenda March"/>
  <Female fullName="Jackie Sailor"/>
  <Female fullName="Kelly Sailor"/>
</xmi:XMI>

```

</xmi:XMI>

Note here, for example, the transformation of the *Sailor* family's father (Peter) into a male person with *fullName* of Peter Sailor (the third *Male* in the *Persons* model instance's XMI above).

In this paper, we are interested in comparing different LLMs with respect to the quality of generating ATL model transformation code. To do so, we have created a basic prompt shown in *Appendix A* which requests from the LLM to generate the ATL model transformation code for the *Families2Persons* example. There are only two input files uploaded to the LLM: *Families.ecore* and *Persons.ecore*. The last two sentences in the prompt capture the transformation rules. Note that the prompt does not state anything about general knowledge facts such as sons and fathers being males while daughters and mothers being females.

IV. APPROACH EVALUATION

In our experiments, we used the following LLMs: Claude from Anthropic (Claude Sonnet 4.5), ChatGP from OpenAI (GPT-5) with *Thinking* mode enabled, and DeepSeek from DeepSeek (V3.2-Exp) with *DeepThink* enabled. The links to the chats are as follows: Claude², ChatGP³, and DeepSeek⁴. We have also created a Github repository in which we uploaded the output ATL model transformation code as well as the output model instances created when executing the ATL code using a local Eclipse development environment⁵.

To evaluate the generated ATL model transformation code, first we check that it compiles correctly. Any compilation error will be detected by the ATL compiler. Compilation errors include syntactic errors, undefined metamodel elements, type mismatches, and unresolved Rule Reference. After compiling the ATL code successfully, we use the ATL virtual machine (VM) to execute the compiled transformation and generate the output model instances. Since we are using an EMF-based launcher with validation enabled, it is guaranteed that the output model conforms to the output metamodel. However, we still need to validate the generated *Persons* model instances for logical errors such as wrong full name or gender for a person. This is done manually by utilizing the *Families* model instance from Section III with two families: *March* and *Sailor*.

The following summarizes the main findings of the experiments:

- 1) The model transformation code generated by Claude and OpenAI compiles successfully and generates the corresponding output model instance.
- 2) The model transformation code generated by DeepSeek has compilation problems, including not using *endif* to close an *if*-statement and the use of *null* instead of *OclUndefined* for testing the definition of references.

- 3) The model transformation code generated by Claude and ChatGPT is well-documented and clear. *Appendix B* includes the model transformation code generated by Claude, as an example.

- 4) The full name is formatted differently between Claude and ChatGPT. For example, *Jim March* is named *March*, *Jim* by Claude's model transformation and *Jim March* by ChatGPT. An instruction concerning formatting can be added in the prompt to ensure desired formatting.

V. CONCLUSION

This study is an early attempt at investigating how contemporary LLMs handle ATL code generation from a minimal natural-language set of prompts and metamodel context. On the evaluated system, Claude and ChatGPT produced compatible transformations that were conformed with the target models, though with some formatting differences, while DeepSeek demonstrated syntactic issues. These observations suggest that (a) general purpose LLMs have internalized ATL idioms despite limited DSL-specific training data; (b) prompt specificity affects semantic adequacy; and (c) simple ATL static checks and validation sets adequately and sufficiently vet outputs. The future work will focus on scaling to multi-rule transformations, exploring alternative systems and testing broader prompting strategies with stronger quantitative metrics. These changes have potential to exhibit LLMs from ad-hoc assistants to reliable authors of MDE transformations.

REFERENCES

- [1] Fr   ric Jouault and Ivan Kurtev. Transforming models with ATL. In *Proceedings of the MoDELS Satellite Events*, pages 128–138. Springer, 2006.
- [2] Mark Chen, Jerry Tworek, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [3] Yizheng Jiang, Xiang Li, et al. A survey on large language models for code generation. *ACM Computing Surveys*, 56(3), 2023.
- [4] Tom Mens and Pieter Van Gorp. A taxonomy of model transformation. *Electronic Notes in Theoretical Computer Science*, 152:125–142, 2006.
- [5] Krzysztof Czarnecki and Simon Helsen. Feature-based survey of model transformation approaches. In *IBM Systems Journal*, volume 45, pages 621–645. IBM, 2006.
- [6] Sergio Mart  nez et al. Reactive model transformation with ATL. *Journal of Object Technology*, 12(3):1–26, 2013.
- [7] M. Tisi, D. Di Ruscio, J. Cabot, and A. Pierantonio. Lazy execution of model-to-model transformations. In *Model Driven Engineering Languages and Systems (MoDELS)*, pages 32–46. Springer, 2011.

²<https://claude.ai/share/ccc5204a-9ddf-40b0-898a-2a9a308579ed>

³<https://chatgpt.com/share/68de679a-cfd0-800c-b0a4-13600522bbc5>

⁴<https://chat.deepseek.com/share/yidhnxw3h4kkv3ti>

⁵https://github.com/ialazzon/ACIT2025_LLMs_ATL

APPENDIX A THE PROMPT

Consider the Families and Persons meta-model ecore files. Generate ATL code to transform Families model instances to Persons instances. Every member in Families that is not female must be transformed to male. Every member in Families that is female must be transformed to female.

[Families.ecore](#)
[Persons.ecore](#)

APPENDIX B ATL CODE GENERATED BY CLAUDE

```
module Claude;

-- @path Families=/Families2PersonsATL/Families.ecore
-- @path Persons=/Families2PersonsATL/Persons.ecore

create OUT: Persons from IN: Families;

-- Transform father to Male
rule Father2Male {
  from
    m : Families!Member (
      not m.familyFather.oclIsUndefined()
    )
  to
    p : Persons!Male (
      fullName <- m.familyFather.lastName + ', ' + m.firstName
    )
}

-- Transform mother to Female
rule Mother2Female {
  from
    m : Families!Member (
      not m.familyMother.oclIsUndefined()
    )
  to
    p : Persons!Female (
      fullName <- m.familyMother.lastName + ', ' + m.firstName
    )
}

-- Transform son to Male
rule Son2Male {
  from
    m : Families!Member (
      not m.familySon.oclIsUndefined()
    )
  to
    p : Persons!Male (
      fullName <- m.familySon.lastName + ', ' + m.firstName
    )
}

-- Transform daughter to Female
rule Daughter2Female {
```

```
from
  m : Families!Member (
    not m.familyDaughter.oclIsUndefined()
  )
to
  p : Persons!Female (
    fullName <- m.familyDaughter.lastName + ', ' + m.firstName
  )
}
```